CODEALPHA

PYTHON PROGRAMMING INTERNSHIP

PROJECT REPORT

TASK 1: HANGMAN GAME

A Simple Text-Based Word Guessing Game Using Python

Submitted By:

Saraswati V. Kulkarni

Branch:

Computer Science Engineering

Internship Domain:

Python Programming

Organization:

CodeAlpha

Academic Year:

2026

Project Type:

Internship Task – 1

HANGAMA GAME:

Hangman Game is a basic console-based Python application in which the player attempts to guess a randomly selected hidden word.

The program contains five predefined words stored in a Python list. At the beginning of the game, one word is selected randomly using the random module. The letters of the selected word are initially hidden from the player.

The player guesses the word by entering one letter at a time through the console. If the entered letter is present in the selected word, the corresponding position is revealed. If the letter is incorrect, the number of remaining attempts is reduced.

DECLARATION

I hereby declare that the project entitled **“Hangama game”** has been developed by me as part of the **CodeAlpha Python Programming Internship – Task 2**.

This project was developed using Python programming concepts and follows the requirements provided for the internship task. The application allows users to enter stock names and quantities, uses a predefined dictionary containing stock prices, and calculates the total investment value.

I declare that the work presented in this report is my own work carried out during the internship period for educational and learning purposes.

Name: Saraswati V. Kulkarni

Branch: Computer Science Engineering

Internship: CodeAlpha Python Programming Internship

Date: 08-08-2026

Place: Banglore

Signature:svk

TABLE OF CONTENTS

1. INTRODUCTION

Hangman is a traditional word-guessing game in which a player attempts to identify a hidden word by guessing one letter at a time.

For this project, the Hangman game has been implemented using the Python programming language. The project follows the simplified requirements provided by CodeAlpha for the Python Programming Internship.

The application is text-based and runs through the console. It uses five predefined words stored in a Python list. At the beginning of a round, one word is selected randomly.

The player then enters letters to guess the hidden word. The program checks each entered letter and provides the appropriate response. Correct guesses reveal letters, while incorrect guesses reduce the number of remaining attempts.

The player is given a maximum of six incorrect guesses. The game ends when the complete word is guessed or the maximum number of incorrect guesses is reached.

This project demonstrates how basic programming concepts can be combined to create a complete and functional application.

2. PROBLEM STATEMENT

The objective of this project is to develop a simple text-based Hangman game using Python.

The application should allow a player to guess a randomly selected word one letter at a time. The program should provide feedback for correct and incorrect guesses and should limit the number of incorrect guesses to six.

The project should use a small list of five predefined words and should not require external files, databases, or APIs.

The application should provide basic console input and output and should implement the game logic using fundamental Python programming concepts.

3. OBJECTIVES

The main objectives of the Hangman Game are:

1. To develop a simple text-based game using Python.
2. To use five predefined words stored in a list.
3. To randomly select a word using the random module.
4. To allow the player to guess the hidden word one letter at a time.
5. To provide a maximum of six incorrect guesses.
6. To identify correct and incorrect guesses.
7. To display the progress of the hidden word.
8. To use a while loop for continuous gameplay.
9. To use if-else statements for decision-making.
10. To practice Python strings and lists.
11. To understand console-based user interaction.
12. To improve logical thinking and problem-solving skills.
13.

4. SCOPE OF THE PROJECT

The scope of this project is to develop a basic console-based Hangman game for educational and entertainment purposes.

The current project is limited to five predefined words and a maximum of six incorrect guesses. The application does not use external files, APIs, databases, graphics, or audio.

The project focuses primarily on demonstrating fundamental Python programming concepts and game logic.

Although the current version has a simple scope, it provides a foundation for developing more advanced versions of the game in the future.

5. TECHNOLOGIES AND TOOLS USED

5.1 Python

Python is the main programming language used to develop the Hangman Game. It provides simple syntax and built-in features that make it suitable for implementing the game logic.

5.2 Random Module

The Python random module is used to select one word randomly from the list of five predefined words.

5.3 Visual Studio Code

Visual Studio Code was used as the development environment for writing, editing, and executing the Python program.

5.4 Windows

The project was developed and tested on a Windows operating system.

5.5 Console

The game uses the Python console for user input and output. The player enters guesses using the keyboard and receives game information through displayed text.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard
- Basic processor
- Sufficient storage space

6.2 Software Requirements

- Windows operating system
- Python 3.x

- Visual Studio Code or another Python IDE
- Python interpreter

6.3 Additional Requirements

No external API, database, file, graphics library, or audio library is required for the basic version of this project.

7. PROJECT DESCRIPTION

The Hangman Game is a console-based Python application in which the player attempts to guess a hidden word.

The application contains five predefined words stored in a list. The random module selects one word from the list for the current game.

The selected word is initially represented using hidden characters. The player enters one letter at a time through the console.

When the player enters a correct letter, the program displays that letter in its appropriate position. If the player enters an incorrect letter, the number of remaining attempts is reduced.

The game continues while the player has remaining attempts and has not yet guessed the complete word.

The player can win by correctly identifying all the letters before reaching six incorrect guesses. If six incorrect guesses are made, the game ends and the correct word can be displayed.

The project demonstrates the practical application of basic Python programming concepts in a simple game.

8. FEATURES OF THE APPLICATION

The main features of the Hangman Game are:

8.1 Five Predefined Words

The program contains a small list of five predefined words.

8.2 Random Word Selection

A word is selected randomly from the list at the beginning of the game.

8.3 Letter-by-Letter Guessing

The player guesses the hidden word by entering individual letters.

8.4 Correct Guess Detection

The program checks whether the entered letter exists in the selected word.

8.5 Incorrect Guess Tracking

Incorrect guesses are counted during the game.

8.6 Six-Guess Limit

The player can make a maximum of six incorrect guesses.

8.7 Word Progress Display

Correctly guessed letters are displayed in their appropriate positions.

8.8 Winning Condition

The player wins when all letters of the selected word have been correctly guessed.

8.9 Losing Condition

The game ends when the player reaches six incorrect guesses without completing the word.

8.10 Console-Based Interaction

The application uses basic text-based input and output through the Python console.

9. METHODOLOGY

The development of the Hangman Game was completed through the following steps:

Step 1: Define the Requirements

The requirements of the CodeAlpha task were analyzed, including five predefined words, random selection, six incorrect guesses, and console interaction.

Step 2: Create the Word List

Five predefined words were stored in a Python list.

Step 3: Select a Random Word

The random module was used to select a word from the list.

Step 4: Hide the Word

The selected word was represented using hidden characters so that the player could not see the complete word initially.

Step 5: Accept Player Input

The program accepts a letter from the player using console input.

Step 6: Check the Guess

The program checks whether the entered letter exists in the selected word.

Step 7: Update the Game

Correct guesses reveal the corresponding letters. Incorrect guesses increase the incorrect-guess count.

Step 8: Check the Result

The program checks whether the complete word has been guessed or whether six incorrect guesses have been reached.

Step 9: Display the Result

A winning or losing message is displayed based on the player's performance.

10. GAME WORKFLOW

The game follows the following workflow:

START

↓

Create list of 5 words

↓

Select a random word

↓

Hide the letters

↓

Display hidden word

↓

Ask player to enter a letter

↓

Check the entered letter

↓

Is the letter correct?

YES → Reveal the letter

NO → Increase incorrect guesses

↓

Check whether the complete word is guessed

↓

YES → Display winning message

NO → Check remaining attempts

↓

If attempts < 6 → Continue guessing

If attempts = 6 → Display losing message

↓

END

11. ALGORITHM

1. Start the program.
2. Import the random module.
3. Create a list containing five predefined words.

4. Select one word randomly from the list.
5. Create a hidden representation of the selected word.
6. Set the incorrect guess count to zero.
7. Start a while loop.
8. Display the current progress of the word.
9. Ask the player to enter a letter.
10. Check whether the guessed letter is present in the word.
11. If the letter is present, reveal the letter.
12. If the letter is not present, increase the incorrect guess count.
13. Check whether the complete word has been guessed.
14. If the word is completely guessed, display a winning message.
15. If the incorrect guess count reaches six, display a losing message.
16. End the game.

12. PYTHON CONCEPTS USED

12.1 Random

The random module is used to select a word randomly from the predefined list.

12.2 Lists

A list is used to store the five predefined words.

Example:

words = ["python", "computer", "program", "hangman", "student"]

12.3 Strings

Strings are used to store words and individual letters entered by the player.

12.4 While Loop

The while loop allows the game to continue accepting guesses until a winning or losing condition is reached.

12.5 If-Else Statements

if-else statements are used to determine whether a player's guess is correct or incorrect and to check the game conditions.

12.6 Variables

Variables are used to store information such as the selected word, guessed letters, and number of incorrect guesses.

12.7 Input and Output

The input() function accepts the player's guess, while print() displays the game status and results.

13. IMPLEMENTATION

The Hangman Game was implemented using Python's basic programming features.

First, a list containing five predefined words is created. The random module is then used to select one word from this list.

The selected word is hidden from the player. A loop is used to repeatedly ask the player for a letter.

For every guess, the program checks whether the entered letter occurs in the selected word. If the letter is correct, the corresponding position is revealed. If the letter is incorrect, the incorrect-guess counter is increased.

The game continues until the player guesses the entire word or reaches six incorrect guesses.

The implementation is intentionally kept simple so that the project demonstrates the fundamental concepts specified in the CodeAlpha task.

14. TESTING AND RESULTS

Testing was performed to verify that the Hangman Game works correctly under different conditions.

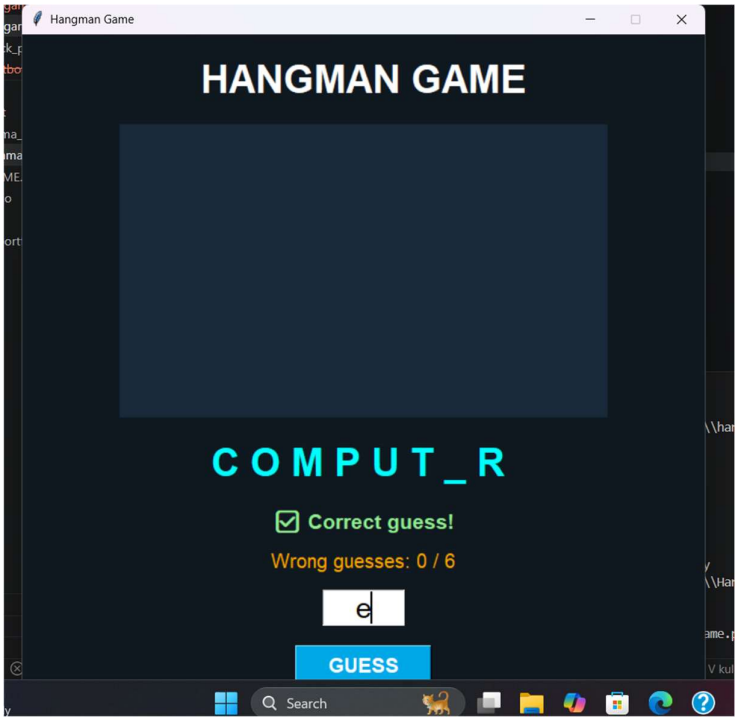
Test Case	Input/Action	Expected Result	Result
1	Start program	Game starts successfully	Pass
2	Correct letter	Letter is revealed	Pass
3	Incorrect letter	Incorrect count increases	Pass
4	Guess complete word	Winning message displayed	Pass
5	Six incorrect guesses	Losing condition displayed	Pass
6	Random word selection	One word selected from list	Pass
7	Multiple guesses	Game continues until result	Pass

The testing results indicate that the major requirements of the Hangman Game were successfully implemented.

15. OUTPUT SCREENSHOTS

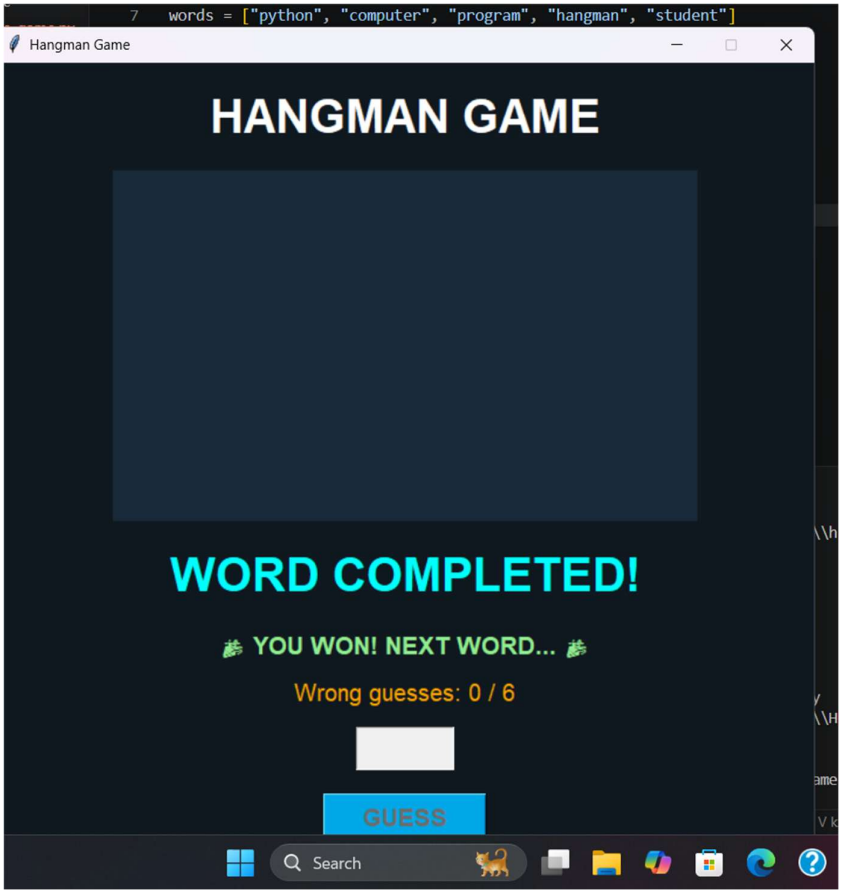
15.1 Game Starting Screen

Figure 1: Hangman Game – Starting Screen



The above screenshot shows the Hangman Game after starting the Python program. The player is provided with the initial hidden word and is asked to enter a letter.

15.2 Correct Guess:



The screenshot shows the result when the player enters a letter that is present in the selected word. The correctly guessed letter is revealed.

15.3 Incorrect Guess

Figure 3: Incorrect Letter Guess

The below screenshot shows the result of an incorrect guess. The number of incorrect guesses is increased.

Fig:3

16. ADVANTAGES

The main advantages of the Hangman Game are:

- Simple and easy to understand.
- Uses basic Python programming concepts.
- Easy to execute and test.
- Does not require an internet connection.
- Does not require external APIs or databases.
- Uses a small predefined word list.

- Provides interactive console-based gameplay.
- Helps beginners understand loops and conditional statements.
- Improves logical thinking and problem-solving skills.
- Can be easily modified and extended.

17.LIMITATIONS

The current version of the Hangman Game has the following limitations:

1. Only five predefined words are available.
2. The game uses a basic console interface.
3. There are no graphics or animations.
4. There are no audio effects.
5. There is no external word database.
6. The game does not maintain a permanent high-score system.
7. There are no difficulty levels.
8. The game does not support multiplayer functionality.

18. FUTURE ENHANCEMENTS

The Hangman Game can be enhanced in several ways in the future.

18.1 Larger Word Collection

More words can be added to increase the variety of the game.

18.2 Difficulty Levels

Easy, Medium, and Hard difficulty levels can be introduced.

18.3 Score System

A scoring system can be implemented to reward players based on their performance.

18.4 Graphical Interface

A graphical user interface could be added in a future version to improve the visual experience.

18.5 Sound Effects

Sound effects could be added for different game events.

18.6 Categories

Words could be organized into categories such as animals, countries, technology, sports, and programming.

18.7 High Score

A high-score system could be added to track the player's best performance.

18.8 External Word Database

A larger word database could be used instead of relying only on five predefined words.

19.CONCLUSION

The **Hangman Game** was successfully developed using Python as part of the **CodeAlpha Python Programming Internship – Task 1**.

The project successfully implements the main requirements of the task, including five predefined words, random word selection, letter-by-letter guessing, a maximum of six incorrect guesses, and basic console input and output.

Through this project, practical knowledge of Python programming concepts such as lists, strings, the random module, while loops, if-else statements, variables, and input/output operations was gained.

The development of the project also helped improve logical thinking, debugging ability, and problem-solving skills.

Although the current project is simple, it provides a strong foundation for developing more advanced games and Python applications in the future.